

# NIZAM Gap Analysis

Product, architecture, operations, security, verification, and knowledge-graph readiness based on the local repository and its existing Graphify graph.

**3.3.0**

Declared platform version

**5,423**

Graph nodes

**11,777**

Graph edges

**2026-06-13**

Assessment date



LIMITED PILOT ONLY

## Executive Verdict

NIZAM has a broad architecture, meaningful governance controls, and a healthy scoped test suite, but the product is not production-ready as a coherent end-to-end system. The largest blockers are a broken canonical repository path, a deterministic agent stub in the live relay path, unresolved and contradictory go-live/security contracts, and missing production validation.

The strongest near-term move is consolidation: establish one canonical tree, reconcile policy drift, wire one real end-to-end persona flow, and make the same verification command pass locally and in CI.

## Assessment Basis

This is a gap analysis, not a compliance certification. Findings combine deterministic repository checks with Graphify topology. Confidence is highest where runtime output and source declarations agree.

### Core Product

**2.0 / 5**



### Verification

**3.0 / 5**



Transport and routing exist; response synthesis remains stubbed.

136 scoped tests pass; default repository collection fails.

## Operations

1.5 / 5



Go-live gates and persistence lift remain on hold.

## Security

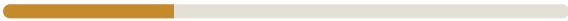
2.0 / 5



Strong intent, but unresolved key rotation and policy conflicts remain.

## Integrations

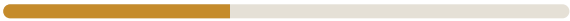
1.5 / 5



Several durable layers are declared unconfigured or incomplete.

## Reproducibility

2.0 / 5



Dependency ranges are floating across all detected requirement sets.

## Documentation

2.5 / 5



Rich documentation, with material state and contract drift.

## Knowledge Graph

2.0 / 5



Broad coverage, but fragmented topology and weak query precision.

# Priority Gaps

## 1. Canonical repository contract is broken

P0

The operator instructions require `D:\NIZAM\nizamcore`, while the actual source and pointer resolve to `D:\NIZAM`. The prescribed verification script halts before checking the product.

- **Impact:** automation, onboarding, deployment receipts, and agent instructions do not share one source of truth.
- **Closure:** choose the canonical tree, update the pointer and scripts atomically, then make the documented startup command executable.
- **Acceptance:** both canonical commands complete successfully from a clean shell without fallback logic.

**Evidence:** `scripts/verify-nizamcore.ps1` returned "Repo not found: D:\NIZAM\nizamcore"; the required interpreter path was absent; `NIZAMCORE_PATH.txt` contains `D:\NIZAM`.

## 2. The end-user intelligence path is still a stub

P0

The relay can authenticate, deduplicate, route, govern, log, and reply, but the coordinator returns canned deterministic output instead of invoking a real persona/model runtime.

- **Impact:** the core product promise is not represented by the tested end-to-end path.
- **Closure:** wire one production-grade persona path behind the existing governor, with timeout, retry, cost, and failure envelopes.
- **Acceptance:** G5.1-G5.3 pass with non-stub replies, traceable model calls, ledger writes, and deterministic safety behavior.

**Evidence:** `NIZAM__system/relay/coordinator.py:47` defines `_agent_stub`; line 110 invokes it. `NIZAM__system/relay/poller.py:17` states that the coordinator is still a deterministic stub.

## 3. Production go-live and security contracts are unresolved

P0

The v3 architecture is explicitly not built, G4/G5 are pending, relay persistence is not lifted, and key rotation remains open. In addition, the v3 narrative retires the AHEL cloud hard-block while the P8 G4 plan still requires it.

- **Impact:** there is no authoritative privacy behavior to validate, and no evidence that the deployed system survives reboot or meets end-to-end safety expectations.
- **Closure:** issue one signed architecture decision record for AHEL/VPS/Drive handling, rotate the exposed key, update tests and runbooks, then execute G4/G5.
- **Acceptance:** all G4 tests pass under one policy; all G5 flows pass; reboot recovery succeeds; key-rotation evidence is recorded without storing the secret.

**Evidence:** `NIZAM__system/hermes-config/DEPLOY_MANIFEST.md:32` says v3 is not built; the same manifest lists G4/G5, relay lift, and OpenRouter key rotation as incomplete. The P8 plan requires an AHEL hard-block while the v3 plan describes that model-egress hard-block as retired.

## 4. Connector registry and runtime readiness are incomplete

P1

Drive, Notion, and the LLM runtime are declared unconfigured. Gmail has no adapter. Telegram is marked as having no adapter even though relay code exists, showing that the registry is stale as well as incomplete.

- **Impact:** fallback, replay, and dual-write behavior cannot be trusted as a product capability.
- **Closure:** replace free-text status with machine-verifiable health probes and reconcile Telegram state with the implemented relay.
- **Acceptance:** startup reports configured/reachable/degraded per connector, and integration tests prove retry, dead-letter, and replay behavior.

**Evidence:** NIZAM\_\_system/policies/CONNECTORS.json:88,101,126 mark Drive, Notion, and LLM runtime unconfigured; lines 110 and 118 mark Telegram and Gmail TODO\_no\_adapter.

## 5. The default test command is not a reliable release gate

P1

Scoped live-code tests are healthy, but repository-wide pytest collection fails because archived snapshots contain duplicate module names.

- **Impact:** contributors and CI cannot use the obvious test command as a trustworthy green/red signal.
- **Closure:** add a root pytest configuration that excludes archives and generated environments, then add a CI workflow for startup, unit tests, and policy consistency.
- **Acceptance:** `python -m pytest -q` passes from the repository root in CI and locally.

**Evidence:** root pytest stopped with two import-file-mismatch collection errors under MAKHZAN\_\_archive. Scoped execution over NIZAM\_\_system, MARSAD\_\_flight\_radar, and tools passed 136 tests plus 14 subtests.

## 6. Builds are not reproducible

P1

Every detected requirements file uses lower-bound ranges rather than exact versions or a lock file.

- **Impact:** deployments can change behavior without a source change, complicating rollback and security review.
- **Closure:** adopt a lock strategy, record Python version, and separate runtime from development dependencies.
- **Acceptance:** clean-environment installation produces the same resolved dependency set and passes startup plus tests.

**Evidence:** the startup receipt reports floating versions in all three detected requirement sets: governor, MARSAD, and the DCF skill.

## 7. Documentation and configuration have material drift

P1

Several operational claims no longer match source state. This creates false blockers and weakens the value of otherwise extensive documentation.

- **Impact:** operators may perform unnecessary changes or validate against retired behavior.
- **Closure:** generate status sections from canonical JSON and add cross-file invariant tests.
- **Acceptance:** no contradictory values for canonical path, dead-letter registration, persona ownership, max turns, or privacy tier behavior.

**Evidence:** `CONNECTORS.json:45` says `DEAD_LETTER` is not declared, but `NIZAM_TEMPLE.json:195` already declares it. The deploy manifest also records `agent.max_turns: 7` as conflicting with `v3`.

## 8. The knowledge graph is broad but not yet decision-grade

P2

The graph captures the corpus, but its topology is fragmented and generic schema tokens dominate centrality. A broad gap query anchored on the word "What" and returned an unrelated template neighborhood.

- **Impact:** GraphRAG answers can be confidently irrelevant, especially for abstract product questions.
- **Closure:** normalize generic schema nodes, down-weight section boilerplate, add domain aliases, and create retrieval regression questions.
- **Acceptance:** a fixed benchmark of architecture and operations questions returns relevant source neighborhoods with cited locations.

**Evidence:** 5,423 nodes, 11,777 edges, 253 communities, and 1,908 nodes with degree 1 or less. Top "god nodes" include generic values such as `ref_null`, `ref_number`, and `ref_string`. The graph report records zero extraction tokens and visible encoding artifacts.

# 30 / 60 / 90 Day Closure Plan

## DAYS 0-30

### Make truth singular

- Resolve canonical path and verification commands.
- Reconcile privacy, persona, dead-letter, and max-turn contracts.
- Exclude archives from pytest and add root CI.
- Rotate the open production key.
- Lock dependencies.

## DAYS 31-60

### Prove one real product loop

- Replace the relay agent stub for one persona flow.
- Add connector health probes and dead-letter replay tests.
- Run G4 in staging under the reconciled policy.
- Add cost, latency, error, and audit dashboards.

## DAYS 61-90

### Earn pilot readiness

- Run G5 end-to-end and measure p95 latency.
- Lift relay persistence and verify reboot recovery.
- Expand real persona paths and failure testing.
- Rebuild Graphify and pass retrieval benchmarks.

## Release Gates

| Gate                  | Required evidence                                                    | Current state                     |
|-----------------------|----------------------------------------------------------------------|-----------------------------------|
| Canonical build       | Documented verify and startup commands pass from the canonical tree  | Blocked by path mismatch          |
| Unit and policy tests | Root pytest passes; cross-file invariants checked in CI              | Scoped green, root collection red |
| Real product loop     | No stub in selected production path; model/cost/audit traces present | Not met                           |
| Security              | One privacy contract; key rotation complete; G4 green                | Not met                           |
| Operations            | G5 green, p95 measured, persistent service survives reboot           | Not met                           |
| Reproducibility       | Locked dependencies and clean install receipt                        | Not met                           |
| Graph quality         | Retrieval benchmark passes with source locations                     | Not met                           |

## Graphify Observations

**Useful signal:** the graph correctly surfaces architecture reports, the go-live runbook, the data model, skills index, workflows index, and protocol index as major bridges. These are the right places to consolidate product truth.

**Risk signal:** 35% of nodes have degree 1 or less, and generic schema references dominate centrality. The current graph should support navigation and evidence discovery, but not autonomous product conclusions without source verification.

## Method and Evidence

---

- Existing Graphify artifacts: `graphify-out/graph.json`, `GRAPH_REPORT.md`, and `.graphify_analysis.json`.
- Canonical verification attempt: `scripts/verify-nizamcore.ps1`.
- Fallback local startup receipt: `.venv/Scripts/python.exe tools/nizam_startup.py`.
- Repository-wide and scoped pytest runs.
- Direct inspection of connector policy, relay coordinator, poller, deployment manifest, v3 plan, P8 plan, requirements, and Git state.
- Graph built from commit `de2b3db9b0bff0c81fea8f4ef5c631f37a223892`; the worktree also contains uncommitted and untracked material, so graph coverage may lag current local state.

Readiness score is an engineering prioritization aid derived from the eight dimensions shown above. It is intentionally conservative and is not a statistical probability.